WEEK-5 PRACTICAL:

Task1:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.



```
  UW PICO 5.09                                File: task1_pipe.c                                Modified

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

#define TOTAL_DATA 1000000

int main() {

    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    // Start measuring time
    struct timeval start, end;
    gettimeofday(&start, NULL);

    if (pid > 0) {
        // ---------------- PARENT: PRODUCER ----------------

        close(pipefd[0]);   // Parent does not read

        int data;

        for (int i = 0; i < TOTAL_DATA; i++) {

            data = i;

            if (write(pipefd[1], &data, sizeof(data)) == -1) {
                perror("write");
                close(pipefd[1]);
                return 1;
            }
        }

        close(pipefd[1]);   // Finished writing

        wait(NULL);         // Wait for child

        gettimeofday(&end, NULL);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1000000.0;

        double total_bytes =
            (double)TOTAL_DATA * sizeof(int);

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Pg    ^K Cut Text      ^C Cur Pos
^X Exit        ^J Justify     ^W Where is     ^V Next Pg    ^U UnCut Text    ^T To Spell
```

```
UW PICO 5.09                          File: task1_pipe.c                          Modified

    close(pipefd[0]);   // Parent does not read

    int data;

    for (int i = 0; i < TOTAL_DATA; i++) {

        data = i;

        if (write(pipefd[1], &data, sizeof(data)) == -1) {
            perror("write");
            close(pipefd[1]);
            return 1;
        }
    }

    close(pipefd[1]);   // Finished writing

    wait(NULL);         // Wait for child

    gettimeofday(&end, NULL);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_usec - start.tv_usec) / 1000000.0;

    double total_bytes =
        (double)TOTAL_DATA * sizeof(int);

    double throughput =
        (total_bytes / (1024.0 * 1024.0)) / time_taken;

    printf("\n===== Communication Efficiency =====\n");
    printf("Data transferred : %.2f MB\n",
           total_bytes / (1024.0 * 1024.0));

    printf("Time taken       : %.6f seconds\n",
           time_taken);

    printf("Throughput       : %.2f MB/s\n",
           throughput);

}
else {
    // ---------- CHILD: CONSUMER ----------

    close(pipefd[1]);   // Child does not write

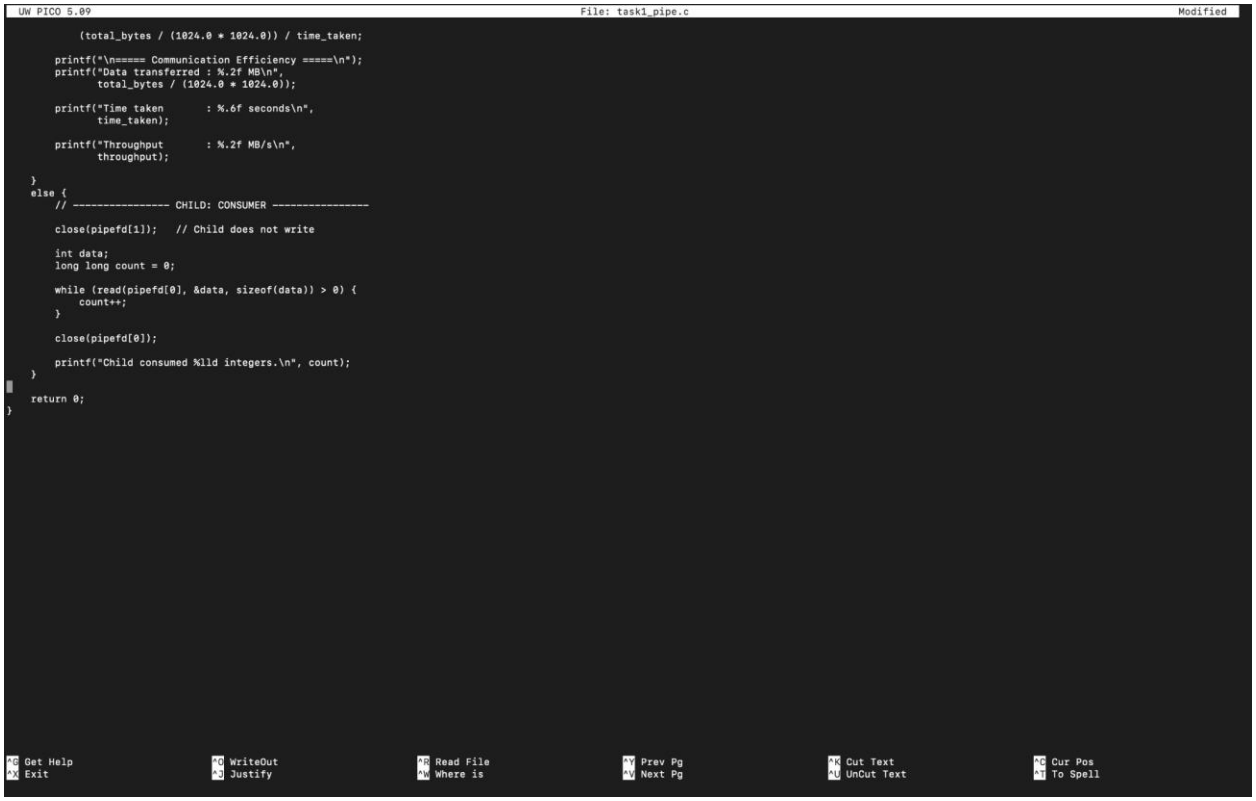
    int data;
    long long count = 0;

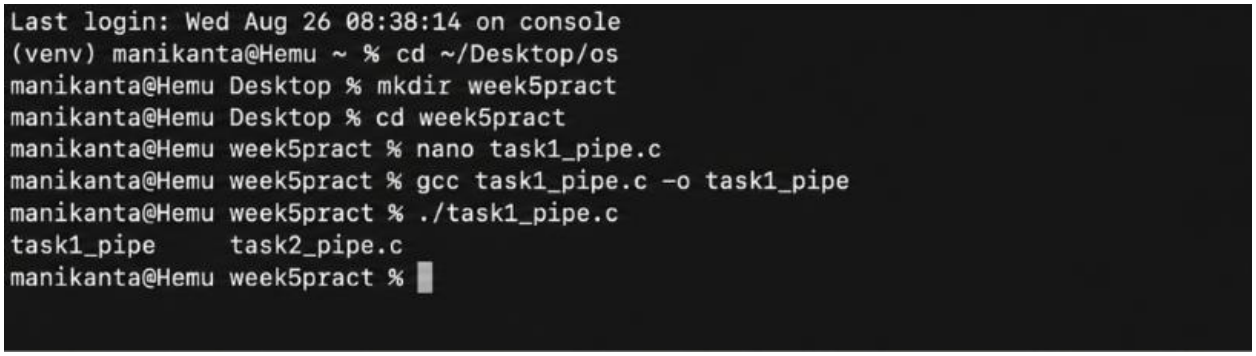
    while (read(pipefd[0], &data, sizeof(data)) > 0) {
        count++;
    }

    close(pipefd[0]);

    printf("Child consumed %lld integers.\n", count);
}

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is     ^V Next Pg    ^U UnCut Text  ^T To Spell
```





Task2:

Develop a program that executes the equivalent of the shell command:

ls -l | grep ".c" using fork() , pipe() , dup2() , and exec()

system calls.

```
 UW PICO 5.09                          File: task2_pipe.c                          Modified

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    int pipefd[2];

    // Create pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    // Create child process
    pid_t pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {

        // --------------- CHILD 1: ls -l ---------------

        close(pipefd[0]);

        // Redirect stdout to pipe
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", (char *)NULL);

        // Only executes if execlp fails
        perror("execlp ls");
        exit(1);
    }

    // Create second child
    pid_t pid2 = fork();

    if (pid2 == -1) {
        perror("fork");
        return 1;
    }

    if (pid2 == 0) {

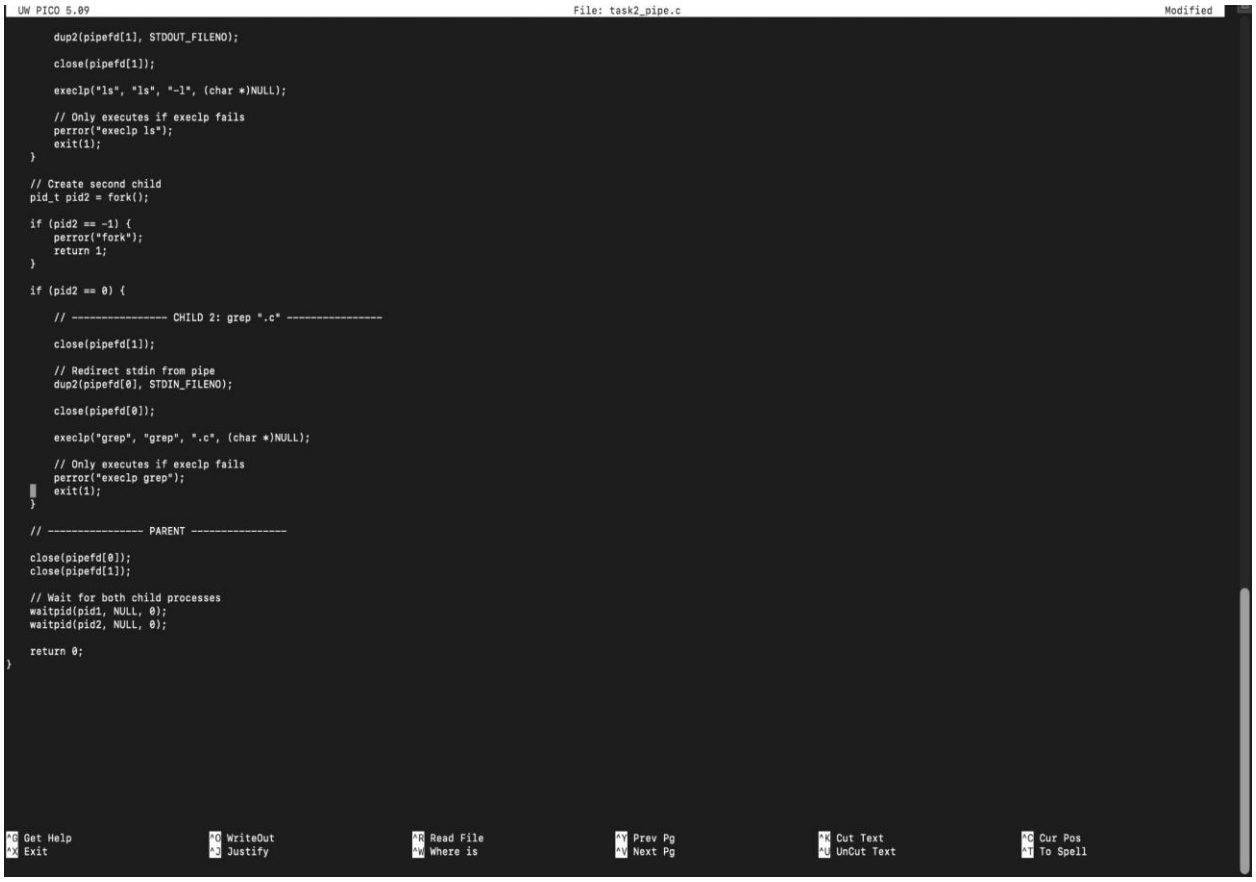
        // --------------- CHILD 2: grep ".c" ---------------

        close(pipefd[1]);

        // Redirect stdin from pipe
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is     ^V Next Pg    ^U UnCut Text  ^T To Spell
```




```
manikanta@Hemu ~ % nano task2_pipe.c
manikanta@Hemu ~ % gcc task2_pipe.c -o task2_pipe.c
manikanta@Hemu ~ % ./task2_pipe.c
-rw-r--r--   1 manikanta  staff    1955 26 Aug 12:14 task1_pipe.c
-rw-r--r--   1 manikanta  staff    1389 26 Aug 12:36 task2_pipe.c
manikanta@Hemu ~ %
```